

DATE: 05/11/2026.

General Purpose Commands:-

* date :

Shows the current date with day, month, time and the year.

OUTPUT: Wed Mar 25

* date + %m (month number)

OUTPUT: 03.

* date + %h (month name).

OUTPUT: Mar.

* date + %y (last 2 digits of year)

OUTPUT: 26.

* date + %H (hours in 24-hour format).

OUTPUT: 14.

* date + %M (minutes).

OUTPUT: 32.

* date + %S (seconds)

OUTPUT: 10

* echo "message"

OUTPUT: message.

* bc - l

* who - show list of users.

* whoami - show login user

* clear - clear terminal screen.

Directory - (commands):-

* pwd - shows current working directory

OUTPUT: /home/teja

* mkdir [directory name].

=> mkdir testdir

* cd testdir (move inside a directory).

* cd.. (move back to Parent directory).

* ls (list all files and directories)

OUTPUT: Documents Downloads testdir

* ls -l

OUTPUT:-

```
drwxr-xr-x 2 teja teja 4096 Mar 25 14:20 testdir
```

```
-rw-r--r--
```

```
1 teja 123 Mar 25 14:10 file.txt
```

* rmdir testdir (delete directory).

File commands:-

* touch [file name] Create an empty file

OUTPUT: touch file1.txt

* cat file1.txt.

OUTPUT: Hello world

Welcome to linux.

* head file1.txt (display first 10 lines) - ③.

OUTPUT : Hello world
welcome to Linux.

* tail file1.txt (display last 10 lines)

* rm file1.txt (remove or delete file).

* mv [old file name] [new file name]

OUTPUT : mv file1.txt newfile.txt.

* mv [file name] [Path].

→ move file to a new location.

OUTPUT : mv newfile.txt /home/teja/documents

* wc -l newfile.txt.

* wc -c newfile.txt.

* wc -w newfile.txt.

File (directory) Permissions:-

d - directory.

r - read.

w - write

x - execute

-- no permission

- rw-r--r-- type user group other.

chmod Symbols:-

U - user.

g - group.

o - other

a - all.

chmod at rwx file . but .

→ gave read, write, execute Perms
to all .

chmod v - rwx file . but

→ remove read and write ps
from user

Octal Method:-

0 - no Permission

1 - execute

2 - write

3 - write and execute

4 - read

5 - read and execute

6 - read and write

7 - read, write and execute

chmod 777 filename

000 - no Permission for user / group or
other

777 - full Permission.

DATE 10/11/26

VI Editor (Visual Editor):-

* vi filename - Open file in vi editor.

OUTPUT: vi file.txt

~
~
~

"file.txt".

* ESC → i (Insert mode).

OUTPUT :- -----INSERT-----

Hello world.

* ESC → :wq (save and Quit)

* ESC → :q! (Quit without saving)

* ESC → o (Insert in next line).

* ESC → shift + o.

* ESC → shift + i

* ESC → shift + a (Insert at end of line)

* x - delete a character.

* z z! - undo all changes.

* ctrl + r - redo.

* shift + v - select whole line.

* y - copy.

Search:-

* Ex: /word

→ search from top, Press n for next match

*ESC : - ? word

→ search from bottom Press n for previous match.

REPLACE:-

*ESC : %s/current/new/g

→ replace all occurrences in file.

*ESC : 1 s/ current/new/g.

→ replace all occurrences in first line

*ESC : 2,4,8 /current/new/g.

→ replace from line 2 to 4.

Line Numbers:-

*ESC: set nu - show line numbers.

*ESC : set nonu - remove line numbers

SED (Stream Editor):-

* sed - stream editor command

* - e - multiple expression

* d - delete.

* g - global.

* P - Point

Print operations:-

⑦.

* sed -n 'sp' data.txt (print 3rd line)

OUTPUT: India.

* sed -n '\$P' data.txt (print last line).

OUTPUT: End.

* sed -n '5, \$P' data.txt (print 3rd to 5th line)

OUTPUT: India

USA

UK.

Delete operations:-

* sed '4d' data.txt.

→ delete 4th line

* sed '\$d' data.txt.

→ delete last line.

* sed '2,6d' data.txt

→ delete line 2 to 6.

NANO Editor:-

* Installation - sudo apt-get update -y & sudo

apt-get install nano-y

* nano demo (open file in nano)

OUTPUT: nano demo.txt

CINU nano b.x demo.txt

* Ctrl + O - save file

* Ctrl + X - exit editor

* Ctrl + K - cut line

* Ctrl + V - Paste line

* Alt + U - undo.

* Alt + R - redo

* Ctrl + W - search

DATE: 20/1/26.

Arithmetic Operations:-

1. /bin /bash

echo "Enter two numbers"

read a

read b.

c = 'expr \$a + \$b'

d = 'expr \$a - \$b'

e = 'expr \$a * \$b'

f = 'expr \$a / \$b'

g = 'expr \$a % \$b'

echo "add \$c"

echo "sub \$d"

echo "mul \$e"

echo "div \$f"

echo "mod \$g"

OUTPUT :- Enter two numbers.

10

5

add 15

sub 5

mul 50

div 2

mod 0.

Basic Shell:-

* If - else :-

```
# 1/bin/bash
```

```
use = 10
```

```
if [ use -ge 18 ]
```

```
then  
    echo "Eligible"
```

```
else  
    echo "Not eligible"
```

```
fi
```

OUTPUT:- NOT eligible.

* If - elif - else:-

```
# 1/bin/bash
```

```
country = "Nepal"
```

```
if [ "$country" = "India" ]
```

```
then
```

```
    echo "you are from India"
```

```
elif [ "$country" = "Nepal" ]
```

```
then
```

```
    echo "country not matched"
```

```
fi
```

OUTPUT:- you are from Nepal.

* while loop:-

```
# 1/bin/bash
```

```
count = 1
```

```
num = 5
```

while [\$count -le \$num]

①.

do

echo "count is \$count"

count=\$((count + 1))

done

OUTPUT :-

count is 1

count is 2

count is 3

count is 4

count is 5

*Functions:-

!/bin /bash

myfun {

echo "Hello from function"

}

myfun

OUTPUT: Hello from function.

fibonacci - series :-

!/bin /bash

echo "Enter number"

read n

a=0

b=1

~~echo~~ "Enter number"

echo " Fibonacci series"
for ((i = 1, i < n, i++))

do
echo \$a
c = \$(a+b)
a = \$b
b = \$c

done

OUTPUT: Enter number

5

Fibonacci series

0

1

1

2

3

Leap Year:-

!bin / bash

echo "Enter number"

read a

m = `expr \$a % 4`

if [\$m -eq 0]

then

echo "Leap year"

else

echo "Not a Leap year"

fi

OUTPUT:- Enter number

2024

Leap Year.

Reverse Number:-

```
# !/bin/bash
```

```
echo "Enter number"
```

```
read n
```

```
rev = 0
```

```
while [ $n -gt 0 ]
```

```
do
```

```
  d = $((n % 10))
```

```
  rev = $((rev * 10 + d))
```

```
done
```

```
echo "Reversed number is $rev"
```

OUTPUT:-

Enter number

1234

Reversed number is 4321

EXP NO: 04

AWK Scripting

DATE:- 27/11/26

Find Average Salary:-

awk'

BEGIN {sum=0; count=0}.

{

sum = sum + \$3

count++

}

END {

Print "Average Salary = ", sum/count

}

'data.txt

OUTPUT:

1 Ram 5000 5

2 Ravi 7000 6.

3 Sita 6000 4

Average Salary = 6000

Using condition (if):-

awk'

{

if (\$5 > 6000)

Print \$2, \$3.

}

'data.txt

OUTPUT:- Ravi 7000.

using Logical AND:-

(15).

awk

```
BEGIN { sum = 0; count = 0; }
```

```
{  
  if ($3 > 6000 & $4 > 40)
```

```
{  
    sum = sum + $3  
    count ++
```

```
}
```

```
}
```

```
END {
```

```
  Print "Filtered Avg salary =" sum / count }
```

'data.txt.

OUTPUT:-

Filtered Avg Salary = 7000

using OR condition:-

awk

```
{  
  if ($3 >= 40 || $4 >= 40)
```

```
    Print $2 - "Pass"
```

```
  else
```

```
    Print $2, "Fail"
```

```
}
```

'data.txt

OUTPUT:-

1	Ram	35	45
2	Ravi	20	30
3	Sita	50	60

Ram Pass

Ravi Fail

Sita Pass

(16)

Create AWK script file ~

Create file :- vi script.awk

```
BEGIN {
```

```
    Print "Processing File"
```

```
    sum = 0
```

```
}
```

```
{
```

```
    sum += $3
```

```
}
```

```
END {
```

```
    Print "Total Salary =", sum
```

```
}
```

To RUN ~

awk -f script.awk data.txt

OUTPUT ~

1. Ram 5000 5

2. Ravi 7000 6

3. Sita 6000 4

Processing File...

Total Salary = 18000.

DATE: 3/2/26

exec:-

#include <stdio.h>

#include <unistd.h>

int main() {

printf("Before exec() \n");

exec("ls", "ls", NULL);

printf("After exec() \n");

return 0;

3.

OUTPUT:- Before exec()

file 1.txt

file 2.txt

script awk.

fork:-

#include <stdio.h>

#include <unistd.h>

int main() {

if (fork() == 0) {

printf("This is CHILD process \n");

} else {

printf("This is PARENT process \n");

}

printf("pid: %d \n", getpid());

return 0;

3.-

OUTPUT :-

this is child process

PID : 1235

this is PARENT process

PID : 1234.

readdir :-

```
#include <stdio.h>
```

```
#include <dirent.h>
```

```
int main () {
```

```
    DIR *dir ;
```

```
    struct "direct" *entry ;
```

```
    dir = opendir (".");
```

```
    if (dir == NULL) {
```

```
        printf ("cannot open directory\n");
```

```
        return 1;
```

```
    }
```

```
    printf ("Files in directory : \n");
```

```
    while ((entry = readdir (dir)) != NULL) {
```

```
        printf ("%s\n", entry->d_name);
```

```
    }
```

```
    close dir (dir);
```

```
    return 0;
```

```
}
```

Output :- File in Directory.

19

:-

file 1. txt

file 2. txt

Script. awk

a. out

pid :-

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main () {
```

```
    printf("PID : %d\n", getpid());
```

```
    printf("PPID : %d\n", getppid());
```

```
    return 0;
```

```
}
```

OUTPUT :-

PID : 2345

PPID : 2200.

opendir :-

```
#include <stdio.h>
```

```
#include <dirent.h>
```

```
int main() {
```

```
    DIR * dir;
```

```
    dir = opendir(".");
```

```
    printf("Failed to open directory\n");
```

```
    return 1;
```

```
}
```

printf("Directory opened successfully ! \n"); (20)

closedir (dir);

return 0;

}

OUTPUT :-

Directory opened successfully.

DATE: 10/2/26.

1) FCFS:-

#include <stdio.h>

int main() {

int n, i;

printf("Enter no. of Process\n");

scanf("%d", &n);

for (i = 0; i < n; i++) {

printf("Enter AT and BT: \n", i+1);

scanf("%d %d", &AT[i], &BT[i]);

}

CT[0] = AT[0] + BT[0];

for (i = 1; i < n; i++) {

CT[i] = AT[i] + BT[i];

}

for (i = 0; i < n; i++) {

TAT[i] = CT[i] - AT[i];

WT[i] = TAT[i] - BT[i];

}

printf("\n Avg TAT: %.2f\n", totalTAT/n);

printf("Avg WT: %.2f\n", totalWT/n);

return 0;

}

Output:-

(22)

Enter no. of Process : 3

Avg TAT : 6.67

Avg WT : 3.33

2) SJF:-

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, j, time = 0, completed = 0, min,
```

```
    printf("Enter no. of Process : ");
```

```
    while (completed < n) {
```

```
        min = -1;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (AT[i] < time && violated[i] == 0)
```

```
            {
```

```
                if (min == -1 || BT[i] < BT[min]) {
```

```
                    min = i;
```

```
            }
```

```
        }
```

```
    }
```

```
    float totalTAT = 0, totalWT = 0;
```

```
    totalTAT += TAT[i];
```

```
    totalWT += WT[i];
```

```
}
```

```
printf("\n Avg TAT = %.2f\n", totalTAT/n);
```

```
printf("Avg WT = %.2f\n", totalWT/n);
```

```
return 0;
```

```
}
```

OUTPUT:-

Enter no. of process : 4

Avg TAT = 9.25

Avg WT = 5.50

2) Round Robin:-

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, tq, i, time = 0, completed = 0;
```

```
    printf ("Enter process : \n");
```

```
    scanf ("%d", &n);
```

```
    printf ("Enter TQ : \n");
```

```
    scanf ("%d", &tq);
```

```
    while (completed < n) {
```

```
        int done = 1;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (RT[i] > tq) {
```

```
                time += tq;
```

```
                RT[i] -= tq;
```

```
            }
```

```
        }
```

```
        if (done)
```

```
            time ++;
```

```
    }
```

```
    totalTAT += TAT[i];
```

```
    totalWT += WT[i];
```

```
}
```

```
printf("total TAT/n");
```

```
printf("total WT/n");
```

```
return 0;
```

3.

OUTPUT :-

Enter Process : 3

Enter TQ : 2

6.67

3-00.

DATE: 17/2/26

#include <stdio.h>

int main() {

int P, n, i, j, k;

printf ("Enter Process: \n");

scanf ("%d", &P);

printf ("Enter resource types: \n");

scanf ("%d", &n);

int alloc[P][n], max[P][n], need[P][n];

int finish[P], safe seq[P];

for (j=0; j<n; j++)

need[i][j] = max[i][j] - alloc[i][j]

for (i=0; i<P; i++)

finish[i] = 0;

int count = 0;

while (count < P) {

int found = 0;

for (i=0; i<P; i++) {

if (finish[i] == 0) {

int Possible = 1;

for (j=0; j<n; j++) {

if (need[i][j] > work[j]) {

Possible = 0;

break;

}

}

}

}

if (!found)

break;

(26)

3.

if (count == p) {

printf("system is in a safe state \n");

printf("safe sequence : ");

for (i = 0; i < p; i++)

printf("P%d", safe_seq[i]);

}

else {

printf("Not in safe state \n");

}

return 0;

3.

OUTPUT:-

Enter process : 5

Enter resource type : 3

system is in a safe state.

safe sequence : P1 P3 P4 P0 P2

EXP NO: 08.

IPC Using Shared memory.
57.

DATE: 24/2/26

1) receiver .c:-

```
#include <stdio.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
int main () {
```

```
    Key .t Key = 1234;
```

```
    int shmId;
```

```
    shmId = shmget(Key, 1024, 0666);
```

```
    char *str = (char*) shmat(shmId, (void*)
```

```
    0, 0);
```

```
    printf("Data read: %s\n", str);
```

```
    shmdt(str);
```

```
    return 0;
```

```
}
```

OUTPUT:-

"Hello from shared memory

Data read: Hello from shared memory -

2) sender .c

```
#include <stdio.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <string.h>
```

```
int main () {
```

key = &key = 1234;

int shmId;

shmId = shmget(key, 1024, 0666 | IPC_CREAT);

char *str = (char *) shmat(shmId, (void *) 0, 0);

printf("Enter message");

scanf("%d [^\n]", str);

printf("Data written : %s \n", str);

return 0;

}

OUTPUT-

Enter message : Hello world.

Data written : Hello world.

DATE: 10/3/26

#include <stdio.h>

#include <stdlib.h>

```
int  mutex = 1, full = 0, empty = 5, x = 0;
int  wait (int s).
```

```
{
    return -s;
```

```
}
int  signal (int s)
```

```
{
    return ++s;
```

```
}
```

void Producer () {

```
    mutex = wait (mutex);
```

```
    empty = wait (empty);
```

```
    full = signal (full);
```

```
    x++;
```

```
    printf ("\\n Producer Producer item %d", x);
```

```
}
```

void Consumer () {

```
    mutex = wait (mutex);
```

```
    full = wait (full);
```

```
    empty = signal (empty);
```

```
    printf ("\\n Consumers item %d", x);
```

```
    x--;
```

```
}
```

int main () {

```
    int n;
```

```
    while (1) {
```

```
printf("\n Enter choice :");
```

```
scanf("%d", &n);
```

```
switch(n){
```

```
case 1;
```

```
if (mutex == 1 || empty != 0){
```

```
    Producer();
```

```
}
```

```
break;
```

```
case 2;
```

```
if (mutex == 1 || full != 0){
```

```
    consumer();
```

```
}
```

```
default;
```

```
    printf("Invalid");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

OUTPUT-

Enter your choice: 1

Producer Producers item: 1

1) FIRST FIT:-

#include <stdio.h>

int main() {

int m, n;

printf("Enter memory blocks\n");

scanf("%d", &m);

int block size [m];

for (int i = 0; i < m; i++) {

scanf("%d", &block size[i]);

}

for (int i = 0; i < n; i++) {

int allocated = 0;

for (int j = 0; j < m; j++) {

int fragment = block size[j] - process[i];

allocated = 1;

break;

}

}

if (!allocated) {

printf("Process %d of size %d is not allocated\n");

}

return 0;

}

OUTPUT:-

Process 1 of size 212 is allocated.

2)

Best Fit ;

(32)

#include <stdio.h>

int main () {

int m, n;

printf("Enter no. of blocks : \n");

scanf("%d", &m);

for (int i=0; i<m; i++) {

scanf("%d", &block_size[i]);

}

for (int i=0; i<n; i++) {

scanf("%d", &process_size[i]);

}

if (bestIndex != -1) {

printf("Process %d of size %d is allocated
to Block %d of size %d with fragment %d \n",

i, process_size[i],

bestIndex, block_size[bestIndex],

process_size[i]);

}

}

return 0;

}

OUTPUT:-

Enter no. of blocks : 5

Process 2 of size 417 is allocated to Block
of size 500 with fragment 83.

DATE: 24/3/26

1) FIFO

#include <stdio.h>

int main() {

int n, frames;

printf("Enter Pages: \n");

scanf("%d", &n);

int ref[n];

for (int i=0; i<n; i++) {

int found = 0;

for (int j=0; j<frames; j++) {

if (frame[j] == ref[i]) {

found = 1;

hits ++;

break;

}

}

if (!found) {

faults ++;

}

}

printf("Total Page Faults: %d\n", faults);

printf("Total Page Hits: %d\n", hits);

}

OUTPUT:- Enter Pages: 7

Total Page Faults: 6

Total page Hits: 1

2)

LRU :-

(2)

#include <stdio.h>

int main() {

int n, frames;

printf("Enter pages : \n");

scanf("%d", &n);

for (int i = 0; i < n; i++) {

int found = 0;

for (int j = 0; j < frames; j++) {

if (frame[j] == ref[i]) {

hits++;

counter++;

time[j] = counter;

found = 1;

break;

}

}

if (!found) {

counter++;

faults++;

}

}

printf("Total Page Faults : %d\n", faults);

printf("Total Page Hits : %d\n", hits);

return 0;

}

OUTPUT:-

Enter Pages : 7

Total Page Faults : 7

Total Page Hits : 0